SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Engineer to Excel

Accredited by NAAC A Grade

SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Introduction

Reverse engineering in software testing means studying the observable behavior of an existing system, its execution traces, outputs, logs, or performance measurements to infer the logic and testing approach behind it. When source code or original test documentation is unavailable, testers use evidence to build a model of the system, identify components and conditions, and design tests that expose defects. This assignment applies black-box techniques such as equivalence class partitioning, boundary value analysis, decision tables, fuzzing, and state-transition testing, together with white-box techniques such as control-flow graphs, branch coverage, and path testing.

The examples and numerical limits used in the answers below are illustrative test-design assumptions where the assignment prompt does not provide exact business rules, source code, API payloads, or logs. In a real project, these assumptions must be confirmed with requirements, product owners, and observed system behavior before tests are treated as authoritative.

1. Reverse Engineering an ATM Withdrawal Module

1.1 Understanding the system and its purpose

The ATM withdrawal module accepts a customer account, a withdrawal amount, and authentication information, then either dispenses cash and updates the account balance or displays an error. The assignment states that only input-output behavior is known, including withdrawal limits, balance validation, and error messages. Therefore, the most suitable starting point is black-box testing: infer the rules from observed inputs and outputs without relying on the internal implementation.

1.2 Decomposition into functional components

• Authentication and session validation: checks that the card/session is valid and the customer has authenticated.

• Input validation: checks that the amount is numeric, positive, and in an accepted denomination.
• Withdrawal-limit validation: checks the per-transaction and daily withdrawal limits.
• Balance validation: confirms that the available balance is sufficient, including any minimum balance rule.
• Cash availability: checks whether the ATM has enough cash and suitable denominations.
• Transaction processing: debits the account and records the transaction only when all required checks pass.
• Response and receipt: displays success or an appropriate error and optionally prints a receipt.

1.3 Equivalence class partitioning

Equivalence class partitioning (ECP) divides inputs into groups expected to behave similarly. One representative from each class is selected to reduce redundant testing.

Amount classes may include: valid positive amount within the limits; zero amount; negative amount; non-numeric input; amount below the minimum; amount above the per-transaction maximum; amount above the remaining daily limit; amount not matching supported denominations; and an amount greater than the available balance. Account classes include valid active account, invalid account, blocked account, and account with insufficient funds. ATM cash classes include sufficient cash and insufficient cash.

1.4 Boundary value analysis

Boundary value analysis focuses on values at and around a limit because errors often occur at comparison boundaries. If, for illustration, the minimum withdrawal is ₹100, the per-transaction maximum is ₹20,000, and the remaining daily limit is ₹40,000, tests should include ₹99, ₹100, ₹101; ₹19,999, ₹20,000, ₹20,001; and amounts just below, equal to, and just above the remaining daily limit. The actual values must be replaced by the bank's specified limits. Test available balance B with amounts $B-1$, B, and $B+1$, while considering denomination restrictions.

1.5 Reconstructed test cases

Test 1: Valid card, sufficient balance, valid amount within all limits. Expected: cash is dispensed, balance is debited once, and success is shown.
Test 2: Amount is zero. Expected: validation error; no debit.
Test 3: Amount is negative. Expected: rejection; no debit.
Test 4: Amount contains letters or invalid symbols. Expected: input validation message.
Test 5: Amount is one unit below the minimum. Expected: reject.
Test 6: Amount equals the minimum. Expected: accept if all other conditions pass.
Test 7: Amount equals the maximum transaction limit. Expected: accept if policy allows the limit inclusively.
Test 8: Amount exceeds the maximum by one unit. Expected: reject with a limit message.
Test 9: Amount equals available balance. Expected: result follows the documented minimum-balance rule.
Test 10: Amount exceeds available balance by one unit. Expected: insufficient-funds message and no debit.
Test 11: Daily limit has already been reached. Expected: reject further withdrawal.
Test 12: ATM does not have enough cash. Expected: informative error and no account debit.

Test 13: Network interruption occurs after the request is submitted. Expected: transaction status is reconciled and the account is not incorrectly debited twice.
Test 14: User presses confirmation twice or retries. Expected: one transaction only, supported by a unique transaction identifier or equivalent protection.
Test 15: Invalid PIN or blocked card. Expected: authentication failure and no transaction.

1.6 Likely original technique and adequacy

The observed limit checks and error messages suggest that the original tester may have used equivalence partitioning and boundary value analysis. If only normal successful withdrawals were tested, adequacy would be low because invalid inputs, exact limits, ATM cash shortages, network failures, and duplicate requests would remain uncovered. A good assessment should report requirements coverage, class coverage, boundary coverage, and transaction-integrity coverage rather than simply counting test cases.

1.7 Conclusion

The improved strategy combines ECP and BVA with negative testing, error-message checks, and transaction-integrity tests. Each test should record preconditions, input, expected output, actual output, and pass/fail status. The most critical acceptance condition is that rejected transactions must not cause an incorrect or duplicate debit.

2. Reverse Engineering White-Box Testing from Execution Traces

2.1 Objective

When code snippets and execution traces exist but test documentation is missing, the tester can reconstruct the white-box approach by identifying statements, decisions, loops, and independent execution paths. The assignment does not provide a specific code listing, so the following small example is a conceptual model rather than a claim about the unseen program.

2.2 Example logic

Assume a function processes an order: it first checks whether the order is valid; if valid, it loops through the items; each item is checked for stock; if all items are available, payment is attempted; otherwise an error is returned. This logic contains multiple decision outcomes and a loop, making it suitable for control-flow analysis.

2.3 Conceptual control-flow graph

Start → Validate order → [Order valid?]
No → Return validation error → End
Yes → Initialize item index → [More items?]
Yes → Check item stock → [Item available?]
No → Return stock error → End
Yes → Move to next item → return to [More items?]
No → Process payment → [Payment accepted?]
No → Return payment error → End
Yes → Confirm order → End

Each rectangle represents a processing step and each diamond represents a decision. A control-flow graph (CFG) makes the possible flow of execution explicit and helps identify branches that may not be exercised by existing tests.

2.4 Decision points and independent paths

The decisions are order validity, whether more items remain, item availability, and payment acceptance. A practical basis set of paths is:
Path 1: invalid order → validation error.
Path 2: valid order → zero items or no remaining items → payment path, if zero-item orders are allowed.
Path 3: valid order → item available → loop completes → payment accepted → order confirmed.
Path 4: valid order → item unavailable → stock error.
Path 5: valid order → all items available → payment declined → payment error.
Path 6: valid order → multiple available items → loop executes more than once → payment accepted.
The exact paths depend on the actual code and business rules. Infinite loop repetitions cannot all be enumerated, so loop testing typically checks zero, one, and multiple iterations, plus a practical upper-limit case.

2.5 Coverage criteria

Statement coverage measures whether each executable statement runs at least once. Branch coverage measures whether each decision outcome, such as true and false, is executed. Condition coverage checks each individual Boolean condition as both true and false. Path coverage checks execution paths, but complete path coverage is often impossible when loops create an unbounded number of paths. Modified Condition/Decision Coverage (MC/DC) may be appropriate for safety-critical logic with compound decisions.

A suite can achieve 100% statement coverage while missing a false branch or an important combination of conditions. Therefore, statement coverage alone does not establish adequate testing.

2.6 Reconstructed test strategy

Create tests for invalid and valid orders, available and unavailable stock, accepted and declined payment, and zero/one/multiple item loop iterations. For each decision, document the expected true and false outcomes. Add tests for exceptions or database failures if these are present in the implementation. Use instrumentation or a coverage tool to compare actual execution against the CFG, and link uncovered edges to additional test cases.

2.7 Evaluating adequacy

The reconstructed suite is adequate only if every identified decision outcome is exercised and all important loop behaviors are tested. Record statement, branch, condition, and path-basis coverage separately. A report should state which paths were covered, which were infeasible, and why. Coverage is evidence of test execution, not proof that the software is defect-free.

2.8 Conclusion

Reverse engineering white-box tests requires translating code and traces into a CFG, identifying decision outcomes, and selecting a manageable set of independent paths. Combining branch coverage with loop testing and boundary-oriented inputs provides stronger evidence than statement coverage alone.

3. Reverse Engineering a Web Login and Transaction System

3.1 System decomposition

A web application containing login and transaction functions should be divided into authentication, session management, authorization, account or wallet retrieval, transaction validation, transaction execution, persistence, and user-facing response components. Separating these components helps classify defects and assign targeted tests.

3.2 Inferring techniques from defect patterns

Repeated failures at minimum or maximum input values may indicate weak boundary value analysis. Defects involving valid and invalid usernames or passwords may indicate equivalence classes were not fully tested. Failures that appear only after login, logout, timeout, or repeated requests may indicate gaps in session-state or path testing. Defects caused by unauthorized users reaching transaction actions may suggest missing negative authorization tests or untested control-flow branches. Duplicate charges after retries may reveal insufficient idempotency and failure-recovery tests.

Defect logs provide clues but do not prove which testing technique was originally used. A tester should phrase conclusions as hypotheses and validate them against test records, code coverage, and reproducible behavior.

3.3 Equivalence classes for login

Valid username and valid password; valid username with incorrect password; unknown username; blank username; blank password; malformed input; disabled or locked account; expired credentials; and excessive failed attempts. Test both the message and security behavior. Error responses should avoid revealing whether a particular username exists if that would create an account-enumeration risk.

3.4 Boundary tests for transactions

For a transaction amount, test zero, negative, minimum allowed, one unit below minimum, maximum allowed, one unit above maximum, available balance minus one, exact available balance, and available balance plus one. If a daily limit applies, test just below, equal to, and just above the remaining limit. Verify currency precision, rounding, and supported decimal places. Actual numeric limits must come from requirements, not assumptions.

3.5 White-box path tests

Model the login flow as: receive credentials → validate format → find account → verify password → check account status → create session → return success. Paths should cover malformed input, unknown account, wrong password, locked account, expired account, and valid

login. The transaction flow should include validation failure, authorization failure, insufficient balance, successful debit and record creation, database failure, payment-provider rejection, timeout, and retry.

3.6 Integrated test matrix

Case A: valid credentials and active account. Expected: authenticated session created.
Case B: invalid password. Expected: authentication denied and no session.
Case C: locked account with correct password. Expected: access denied according to policy.
Case D: unauthenticated request to transaction endpoint. Expected: reject and do not process.
Case E: authenticated user attempts another user's transaction. Expected: authorization failure.
Case F: valid transaction within limits. Expected: exactly one debit and transaction record.
Case G: amount exceeds available balance. Expected: reject without changing balance.
Case H: provider timeout after request. Expected: query status before retry; do not create a duplicate transaction.
Case I: simultaneous requests using the same balance. Expected: concurrency control prevents overspending.
Case J: database error while saving. Expected: transaction is rolled back or reconciled; no inconsistent partial result.
Case K: session expires before confirmation. Expected: require re-authentication or reject according to policy.
Case L: SQL injection-like input in login or filter fields. Expected: parameterized processing and safe rejection or literal treatment.

3.7 Coverage gaps and improvements

The strategy should combine ECP and BVA for input rules, decision tables for combinations such as authenticated/authorized/sufficient-funds/within-limit, and path testing for the internal login and transaction flow. Add security testing, concurrency testing, recovery testing, and audit-log verification. Measure requirements coverage, branch coverage, and end-to-end transaction outcomes.

3.8 Conclusion

The best reconstructed approach combines black-box tests of inputs and outputs with white-box path tests of critical logic. It also verifies that failures do not create sessions or financial changes improperly, and that successful operations are recorded exactly once.

4. Reverse Engineering a Legacy Inventory Management System

4.1 Understanding the stock update process

The system accepts stock transactions and generates low-stock alerts. A typical update may validate the product identifier and quantity, retrieve current stock, calculate the new stock level, save the update, and compare the result with a reorder threshold. Since the prompt provides logs rather than exact business rules, threshold values and permitted stock behavior must be confirmed before final test execution.

4.2 Functional decomposition

• Product identification: confirm the product exists and is active.
• Quantity validation: ensure the quantity is numeric and within supported limits.
• Transaction type: distinguish stock receipt, sale, return, and adjustment.
• Stock calculation: add or subtract the quantity according to transaction type.
• Persistence: save the stock change consistently.
• Low-stock rule: compare resulting stock with the configured threshold.
• Alert generation: create or suppress an alert and avoid repeated duplicate alerts.
• Audit logging: retain transaction details and the previous and new quantities.

4.3 Equivalence partitions

For quantity input, valid positive quantity; zero; negative quantity; non-numeric value; excessively large quantity; and fractional quantity where only whole units are allowed. For product status, existing active product, unknown product, and inactive product. For stock level, below zero, exactly zero, above zero but below the threshold, equal to threshold, and above threshold. For transaction types, valid receipt/sale/return/adjustment and unknown transaction type.

4.4 Boundary value analysis

Let the configured low-stock threshold be T and current stock be S. Test resulting stock T−1, T, and T+1 to establish whether the alert rule is “below threshold” or “at or below threshold.” Test stock values −1, 0, and 1 if negative stock is prohibited. If the maximum permitted quantity is M, test M−1, M, and M+1. For a sale of quantity Q from stock S, test Q=S−1, Q=S, and Q=S+1 to confirm handling of insufficient stock. The threshold and maximum values are illustrative variables; use the real configuration values in the final test sheet.

4.5 Reconstructed test cases

1. Receive stock for a valid product with a positive quantity: stock increases by the exact quantity and an audit entry is created.
2. Attempt to receive zero quantity: reject or follow the documented no-op rule.
3. Submit a negative receipt quantity: reject unless negative adjustments are explicitly supported through a separate transaction type.
4. Sell one unit when stock is available: stock decreases by one.
5. Sell the exact remaining stock: result follows the documented zero-stock policy.
6. Sell one more unit than available: reject or create a controlled back-order if supported; never silently corrupt stock.
7. Update stock to T−1: low-stock alert should be generated if the rule is stock below T.
8. Update stock to T: verify whether the threshold comparison is inclusive.
9. Update stock to T+1: no low-stock alert if the rule is at or below T.
10. Update an unknown product: reject with no database change.
11. Repeat the same transaction request: detect duplicate transaction ID and avoid double counting.
12. Database fails during update: rollback or reconcile to prevent a mismatch between stock and transaction log.

13. Two sales occur concurrently: locking or atomic update prevents stock becoming incorrectly negative.
14. Stock crosses above the threshold after replenishment: close or clear the alert according to policy.
15. Repeated low-stock updates: alert deduplication prevents a flood of identical notifications.

4.6 Log analysis and adequacy

Successful and failed logs can reveal which inputs are accepted and which thresholds trigger alerts. Compare input quantities, previous stock, resulting stock, alert status, and transaction identifiers. However, absence of a failure in a small log set does not prove that a boundary was tested. Build a coverage matrix showing each equivalence class and boundary, then add missing cases.

4.7 Conclusion

A sound black-box test design covers input classes, threshold boundaries, stock consistency, duplicate transactions, concurrent updates, and alert behavior. The central expected invariant is that the recorded stock must equal the result of valid committed transactions, and failed operations must not leave partial changes.

5. Reverse Engineering a Compiled Library Module from CPU and Memory Profiles

5.1 Goal and limitations

A compiled executable and CPU/memory profile can reveal where the program spends resources, but a profile does not directly expose source-level logic. Execution spikes may suggest loops, repeated searches, sorting, large allocations, or repeated database/file access. These are hypotheses to investigate with controlled inputs and profiling tools, not definitive proof of a particular code structure.

5.2 Conceptual components

For a library management module, likely operations include searching books, issuing or returning items, checking member eligibility, calculating overdue fines, updating records, and generating reports. Resource-heavy operations might include searching a large catalogue, iterating over transaction history, or producing a report for many members.

5.3 Conceptual CFG

Start → Read request → Validate request → [Input valid?]
No → Return error → End
Yes → Load library data → [Operation type?]
Search → Loop through catalogue → [Match found?] → Continue search / return results
Issue or return → Check member status → [Eligible?] → Update record / reject
Report → Loop through records → Aggregate results → Generate report
All successful paths → Save or return result → End
This CFG is conceptual because the actual executable's control flow is not supplied. A real reverse-engineering exercise would combine profiler evidence, debugger traces, symbol information if available, and controlled input experiments.

5.4 Mapping profile spikes to test targets

A CPU spike that grows with catalogue size may indicate an algorithm whose work increases with the number of records. A memory spike during report generation may indicate large temporary collections or unbounded buffering. Repeated spikes for one request may indicate nested loops, retries, or repeated parsing. To investigate, run the same operation with small, medium, and large datasets, holding other variables constant, and compare runtime and peak memory.

5.5 Path testing strategy

Test invalid request; valid search with no match; valid search with one match; search with many matches; eligible member issuing an available book; ineligible member; book already issued; return of an issued book; return of a book not issued; report over an empty catalogue; report over a large catalogue; and errors during file/database access. For loops, test zero records, one record, typical size, high-volume size, and the largest supported input. Add long-running repeated requests to detect leaks or state accumulation.

5.6 Resource and stability tests

Measure CPU time, peak memory, response time, throughput, and error rate. Repeat tests to detect memory leaks. Include sustained load, sudden bursts, and recovery after load. Define acceptance thresholds from the product's service requirements instead of inventing universal limits. Confirm that resource pressure produces controlled errors rather than crashes or corrupted data.

5.7 Coverage evaluation

Without source-level instrumentation, exact branch coverage may not be measurable. The team can still document externally observed paths and use available dynamic analysis tools if permitted. If symbols or instrumentation are available, map execution traces to branches and compare them with the conceptual CFG. Clearly distinguish confirmed coverage from inferred behavior.

5.8 Conclusion

The reconstructed strategy uses profile-guided experiments to prioritize costly paths, while functional tests ensure correct library behavior. Performance and memory evidence should be connected to repeatable test inputs and measurable acceptance criteria.

6. Reverse Engineering a Mobile Payment API State Machine

6.1 Purpose

The API exposes transaction states Pending, Success, Reversal, and Timeout. State-transition testing checks not only individual responses but also whether each transition from one state to another is legal under specified triggers. The exact JSON schemas and triggers are not included in the assignment prompt, so the model below is a structured conceptual reconstruction that must be checked against the real API contract.

6.2 State meanings

Pending: transaction request has been accepted but final outcome is not yet confirmed.
Success: payment completed and the transaction is committed.
Reversal: a previously applied or reserved transaction is reversed or compensated.
Timeout: the system did not receive a final response within the allowed interval. Timeout may be a final state or a temporary status requiring reconciliation, depending on the design.

6.3 Conceptual transition model

Start → Pending
Pending → Success when payment confirmation is received.
Pending → Reversal when a failure requires compensation or a previously reserved amount must be released.
Pending → Timeout when the response deadline expires.
Timeout → Success if a later authoritative status check confirms the payment succeeded.
Timeout → Reversal if reconciliation confirms the payment failed and funds must be released.
Success → Reversal only if the business rules permit a later reversal or refund.
Reversal → terminal state, with repeated reversal requests handled idempotently.
Illegal transitions, such as an unsupported direct transition from an initial request to Reversal without a transaction record, should be rejected or safely reconciled.

6.4 State-transition testing technique

The prompt points to implicit state-transition testing. Test coverage can be measured at several levels: state coverage (visit every state), transition coverage (exercise every legal edge), transition-pair coverage (exercise pairs of consecutive transitions), and invalid-transition coverage (attempt representative illegal edges). State coverage alone is insufficient because a test may visit all states while missing an important transition.

6.5 Reconstructed test suite

1. Submit valid payment: state becomes Pending.
2. Receive valid success callback: Pending → Success.
3. Receive failure requiring compensation: Pending → Reversal, if defined by the contract.
4. No response before deadline: Pending → Timeout.
5. Timeout followed by status check confirming success: Timeout → Success, if permitted.
6. Timeout followed by status check confirming failure: Timeout → Reversal or a documented failed terminal status.
7. Duplicate success callback: do not create a second debit or second transaction.
8. Duplicate reversal request: reversal remains idempotent; funds are not released twice.
9. Attempt unsupported transition from Success to Pending: reject or ignore safely.
10. Attempt reversal of a transaction that never existed: reject with a clear API error.
11. Malformed JSON or missing transaction ID: schema validation error; no state change.
12. Callback with mismatched amount or currency: reject and log for investigation.
13. Callback arrives after timeout: reconcile against authoritative transaction status.
14. Two conflicting callbacks arrive concurrently: final state follows a deterministic, documented rule.

15. Retry the same API request with the same idempotency key: return the existing result rather than create another payment.

6.6 Data constraints and response validation

Check transaction identifier format, required fields, allowed state names, amount precision, currency code, timestamp format, and signature/authentication fields where applicable. Validate that each response has a consistent state and that impossible combinations are rejected. Sensitive data should not be exposed in error messages or logs.

6.7 Evaluation

Use a transition table with columns for current state, event, guard condition, next state, and expected response. Mark each legal transition as covered by at least one test and include negative tests for prohibited transitions. Test recovery and delayed callbacks because these often create payment inconsistencies.

6.8 Conclusion

A state-based suite should cover all states, legal transitions, selected transition pairs, illegal transitions, retries, and concurrency. The key safety requirement is that a delayed or duplicated message must not result in duplicate charging or incorrect reversal.

7. Fuzzing and Boundary Testing for an Encrypted Communication Module

7.1 System decomposition

The module accepts parameters and returns encrypted output. It can be divided into input parsing, field validation, message-length handling, key/configuration validation, encryption, output encoding, and error handling. Since encryption output may vary due to random nonces or initialization vectors, testing should normally verify cryptographic properties and protocol structure rather than expecting identical ciphertext for every run.

7.2 Fuzzing approach

Fuzzing supplies unexpected, malformed, extreme, or randomly generated inputs to reveal crashes, hangs, memory errors, invalid parsing, and inconsistent error handling. Begin with a valid seed message, then mutate each field separately and in combinations. Use a controlled test environment and avoid sending malformed fuzz traffic to production systems.

7.3 Input equivalence classes and boundaries

For a text field, classes include empty, valid short text, maximum supported length, overlong text, invalid encoding, embedded null characters, control characters, and unexpected binary data. For numeric fields, test negative values, zero, maximum allowed, and values beyond the maximum. For key or algorithm identifiers, test supported values, unsupported values, missing values, and malformed encodings. If a field allows length L, test L−1, L, and L+1 bytes, not merely character count, because multi-byte encodings may use different numbers of bytes.

7.4 Reconstructed test cases

1. Valid message within the supported size: encryption succeeds and output can be validated/decrypted by the authorized test harness.

2. Empty message: follow the documented empty-input policy.
3. Message at maximum permitted length: process safely.
4. Message one byte over maximum: reject without crash or truncation that changes meaning.
5. Very large input: reject or handle under documented resource limits.
6. Missing required field: return a controlled validation error.
7. Invalid field type: reject safely.
8. Malformed Unicode or binary sequence: parser handles it without memory corruption.
9. Embedded null and control characters: preserve or reject according to the protocol.
10. Invalid key length: reject before encryption.
11. Unsupported algorithm identifier: return a controlled error.
12. Repeated malformed requests: no resource leak or service degradation.
13. Input containing SQL-like or shell-like metacharacters: treat as data; never execute it as code.
14. Random fuzz corpus: no crash, hang, out-of-bounds access, or uncontrolled memory growth.
15. Error path: ensure error responses do not expose secret keys, plaintext, or internal cryptographic details.

7.5 Preventing buffer overflow and injection

Boundary tests help expose length mistakes but cannot alone prove memory safety. Use safe language/runtime features, bounds-checked buffers, length validation before copying, and secure parsing libraries. For injection prevention, do not concatenate untrusted values into commands or queries; use parameterized interfaces and strict allow-lists where appropriate. Encryption does not automatically prevent injection vulnerabilities, and encrypted data should be authenticated using a suitable authenticated-encryption design or a correctly implemented protocol.

7.6 Evaluation and reporting

Record input seed, mutation, expected result, actual result, crash details, resource usage, and reproducibility steps. Classify defects by severity and determine whether they affect confidentiality, integrity, availability, or protocol compatibility. Run sanitizers and memory-analysis tools where available.

7.7 Conclusion

The improved strategy combines ECP, byte-oriented BVA, fuzzing, malformed-input tests, and security checks. Its objective is to ensure that all invalid input is handled safely and that the module never corrupts memory or leaks sensitive information when processing unexpected data.

8. Improving Coverage for an Insurance Premium Calculator

8.1 Why 100% statement coverage can still miss defects

Statement coverage proves that each executable statement ran at least once, but it does not prove that every decision outcome or Boolean condition was tested. For example, a premium rule might depend on age, claim history, and vehicle category. A test may execute the calculation statement while failing to exercise a false outcome for one condition or a specific combination of conditions.

8.2 Illustrative business logic

Assume a premium calculator applies a base premium and then checks: (A) driver age is below a threshold, (B) the customer has prior claims, and (C) the vehicle is in a high-risk category. A surcharge may apply when one or more conditions are true. The exact formula is not provided in the prompt, so the example demonstrates the testing method rather than a real insurance rule.

8.3 Conceptual CFG

Start → Validate input → [Input valid?]
No → Return validation error → End
Yes → Calculate base premium → Evaluate risk conditions → [Surcharge condition true?]
Yes → Add surcharge → [Discount eligible?] → Apply discount or skip discount → Return final premium
No → [Discount eligible?] → Apply discount or skip discount → Return final premium
This graph contains validation, a compound decision, and discount decision. Each edge should be mapped to at least one test.

8.4 Coverage criteria

Statement coverage executes statements. Branch coverage executes true and false outcomes of decisions. Condition coverage ensures each atomic condition becomes true and false. Decision-table testing covers meaningful combinations of conditions and actions. MC/DC requires each condition to independently affect the decision outcome and is useful where compound logic is critical. Complete path coverage may be impractical as the number of combinations grows, but a basis of independent paths and decision combinations can be tested.

8.5 Reconstructed test design

Test 1: all inputs valid, all risk conditions false, discount ineligible.
Test 2: input invalid, such as missing age or invalid vehicle category; expected validation error.
Test 3: age condition true while other conditions are false.
Test 4: claim-history condition true while other conditions are false.
Test 5: high-risk category true while other conditions are false.
Test 6: all three risk conditions true.
Test 7: compound decision false when all component conditions are false.
Test 8: discount eligible and surcharge absent.
Test 9: discount eligible and surcharge present, confirming the documented order of calculation.
Test 10: age exactly at the risk threshold and one unit on either side.
Test 11: premium at minimum and maximum supported values.
Test 12: rounding case with decimal values to verify currency precision.
Test 13: missing or extreme numeric values; expected safe validation.
Test 14: combination where one condition changes and all other conditions remain fixed, demonstrating its independent effect on the decision.
The exact test inputs should use the real threshold and premium formula.

8.6 Coverage reconstruction

Create a condition table listing each Boolean condition and its true/false test. Build a CFG and annotate each edge with test identifiers. Run a coverage tool to determine whether every branch

was executed. For a compound condition, add tests that isolate each condition's influence. If a branch is infeasible because of validated business constraints, document the reason rather than claiming it was tested.

8.7 Risk reduction

Branch and condition-focused tests reduce the chance that a premium is calculated incorrectly for one customer category. Boundary tests catch off-by-one errors at age or premium limits, while decision-table tests expose interactions among claims, age, and risk category. Also test rounding, missing data, and rule precedence.

8.8 Conclusion

The key lesson is that 100% statement coverage is not enough. A stronger suite uses branch coverage, condition coverage, decision tables, and independent paths, with a traceable link between every rule and its tests.

9. Reverse Engineering a Cloud File Synchronization Tool

9.1 Understanding the synchronization problem

A cloud synchronization tool compares local and remote files and transfers changes across a network. The assignment mentions defect reports involving file size, file type, and network speed. These patterns suggest the original equivalence classes may have been too broad or may not have included extreme sizes, unsupported types, interrupted transfers, or slow connections.

9.2 Functional components

- File discovery and metadata reading.
- File-type and permission validation.
- File-size validation and chunking.
- Change detection using metadata, timestamps, or hashes.
- Upload and download scheduling.
- Network transfer and retry.
- Conflict resolution when local and remote versions change.
- Temporary-file handling and final atomic replacement.
- Integrity verification and synchronization logging.

9.3 Equivalence classes

File-size classes: empty file, small file, typical file, maximum supported file, and oversized file. File-type classes: supported type, unsupported type, extension/content mismatch, hidden/system file if relevant, and files with unusual names. Network classes: fast/stable, slow but stable, intermittent, disconnected, and connection restored after interruption. File state classes: unchanged file, local-only update, remote-only update, simultaneous edits, deleted file, and renamed file.

9.4 Boundary value analysis

If the supported maximum file size is M bytes, test $M-1$, M, and $M+1$ bytes. If transfer chunk size is C, test file sizes $C-1$, C, $C+1$, $2C-1$, 2C, and $2C+1$ to expose chunk-boundary defects.

Test empty files and files containing exactly one byte. For network throughput, use stable high speed, near-minimum supported speed, intermittent pauses, and complete disconnection. The actual limits should be taken from product requirements.

9.5 Stress and recovery tests

Transfer many small files, a few very large files, and mixed file types. Run synchronization while bandwidth is limited, disconnect the network mid-transfer, restart the application, and reconnect. Confirm that partial files are not exposed as complete and that the tool resumes or restarts safely. Use checksums or cryptographic hashes to verify byte-for-byte integrity after transfer.

9.6 Reconstructed test cases

1. Upload a supported small file: remote content matches local content.
2. Upload an empty file: zero-byte content is preserved.
3. Upload a file at the maximum size: complete transfer and correct hash.
4. Upload a file above the limit: clear rejection without partial corruption.
5. Transfer file sizes around chunk boundaries: no missing or duplicated bytes.
6. Transfer an unsupported file type: follow the documented policy.
7. Rename a file: remote and local names reconcile without creating accidental duplicates.
8. Delete a file locally: deletion behavior follows the configured synchronization policy.
9. Modify the same file on two devices: conflict is resolved without silently losing a version.
10. Disconnect during upload: incomplete temporary data is not treated as complete.
11. Restore the network: transfer resumes or restarts and validates integrity.
12. Slow network with repeated retries: no unbounded retry loop or duplicate file creation.
13. Fill available storage during download: report the error and preserve existing files.
14. Synchronize many files: memory, CPU, and queue behavior remain within requirements.
15. Corrupt a chunk or checksum: detect corruption and retry or reject the transfer.

9.7 How the strategy prevents data loss

The combination of ECP and BVA tests normal and extreme file conditions. Stress tests reveal failures that occur only with volume or slow networks. Recovery tests exercise interruption, retries, and restarts. Hash verification ensures that successful status corresponds to correct content, not merely a completed network request. Atomic writes and conflict preservation reduce the risk of overwriting a valid file with incomplete or stale content.

9.8 Conclusion

The revised strategy must test file size, type, network conditions, conflict behavior, recovery, and content integrity. The most important acceptance rule is that the system must never silently report success when transferred data is incomplete or corrupted.

10. Reverse Engineering an E-commerce Search and Filter Module

10.1 Objective and assumptions

The module receives search and filter parameters and generates SQL queries, but the source code is unavailable. By comparing query parameters with returned datasets, the tester can infer

whether conditions are combined with AND, OR, or NOT and reconstruct the decision logic. Since the prompt does not include actual SQL statements or datasets, the decision table below is illustrative and should be adapted to the observed queries.

10.2 Decomposition

• Parameter parsing: reads keyword, category, price range, brand, stock status, and sort order.
• Input validation: checks types, allowed values, range ordering, and missing fields.
• Predicate construction: converts selected filters into database conditions.
• Boolean combination: combines predicates using AND, OR, or NOT.
• Query execution: executes the prepared query safely.
• Result ordering and pagination: sorts and limits the dataset.
• Response formatting: returns products and metadata such as result count.

10.3 Inferring AND, OR, and NOT

Suppose the user selects category = “books” and in-stock = true. If results contain only products satisfying both conditions, the logic likely uses AND. If the result includes products matching either condition, the logic may use OR. If a filter excludes a selected category or discontinued items, the query may use NOT or a not-equal predicate. To infer behavior reliably, create controlled records that differ in only one attribute and observe whether each appears in the result.

10.4 Illustrative decision table

Conditions: C1 = keyword matches; C2 = category matches; C3 = price is within range; C4 = item is in stock.
Rule 1: C1 yes, C2 yes, C3 yes, C4 yes → include product.
Rule 2: C1 yes, C2 yes, C3 yes, C4 no → include only if stock filter is not selected; otherwise exclude.
Rule 3: C1 yes, C2 no, C3 yes, C4 yes → exclude when category is mandatory and combined with AND.
Rule 4: C1 no, C2 yes, C3 yes, C4 yes → include if keyword is optional/empty; otherwise exclude.
Rule 5: C1 yes, C2 yes, C3 no, C4 yes → exclude when price filter is active.
Rule 6: keyword matches OR category matches → include if the observed business rule uses OR for these fields.
Rule 7: excluded category is selected → exclude that category when NOT/exclusion logic is enabled.
Rule 8: no filters are supplied → return the default searchable catalogue, subject to access and pagination rules.
The table illustrates why the same conditions can produce different outcomes depending on whether a filter is active, optional, or combined with AND/OR.

10.5 Reconstructed test cases

1. Keyword only with a known matching product: product appears.
2. Keyword with no match: empty result set and correct count.
3. Category only: all returned products belong to that category.
4. Price minimum and maximum: products at the endpoints are handled according to

inclusive/exclusive requirements.
5. Price just below minimum and just above maximum: excluded if range is inclusive.
6. Category AND in-stock: every result satisfies both predicates.
7. Keyword OR category: confirm the intended union of matches if OR is specified.
8. NOT/exclusion filter: excluded products never appear.
9. Conflicting filters: return no results or a validation message according to requirements.
10. Missing filter values: do not create accidental restrictive or permissive conditions.
11. Empty strings, malformed numbers, negative prices, and reversed ranges: reject or normalize safely.
12. Pagination: pages do not omit or duplicate products under stable data.
13. Sorting ascending and descending: order is correct, including equal values.
14. SQL injection-like input: treated as data; parameterized SQL prevents executable input from changing query structure.
15. Large result set and multiple filters: response remains correct and performs within requirements.

10.6 SQL safety and result verification

Use parameterized queries rather than string concatenation. Confirm that user input cannot modify the SQL structure. Compare actual returned product IDs with a manually calculated expected dataset. Verify that result count, page count, sorting, and filters agree. Test null values and missing database attributes where relevant.

10.7 Completeness evaluation

The reconstructed model is complete when each filter can be tested alone, important pairs of filters are tested together, AND/OR/NOT behavior is confirmed, boundaries are tested, and invalid inputs are covered. Exhaustive combinations may grow rapidly; use a decision table and pairwise combinations for lower-risk filters, while retaining full combinations for critical business rules. Keep a traceability matrix from each requirement or inferred rule to its tests.

10.8 Conclusion

Decision-table testing is effective for search logic because it makes combinations and outcomes explicit. Combining it with boundary tests, result-set verification, and SQL-injection resistance provides stronger confidence than checking a few ordinary searches.

Overall Conclusion

This assignment demonstrates how reverse engineering supports software testing when the original test design or implementation details are incomplete. Black-box methods reconstruct behavior from inputs and outputs, including equivalence classes, boundary values, decision tables, fuzzing, and state transitions. White-box methods use code and traces to identify decisions, loops, control-flow graphs, branches, and independent paths. Profile-guided tests, stress tests, recovery tests, and security tests broaden the analysis beyond ordinary functional cases.

A high-quality test suite should state assumptions, expected results, and coverage criteria clearly. Test results should be reproducible and traceable to system rules. Where the assignment provides no exact thresholds, source code, logs, JSON examples, or SQL queries, the conceptual models in this document must be validated against the real system before use.

Code of Conduct

I **Sadhana K/192421137** certify that this submission is my work and that I have followed the academic integrity guidelines applicable to this assessment.

Signature: ***Sadhana K***

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation